Stop Using /init for AGENTS.md

FEBRUARY 23, 2026

TL;DR: A good mental model is to treat AGENTS.md as a living list of codebase smells you haven't fixed yet, not a permanent configuration. Auto-generated AGENTS.md files hurt agent performance and inflate costs by 20%+ because they duplicate what agents can already discover. Human-written files help only when they contain non-discoverable information - tooling gotchas, non-obvious conventions, landmines. Every other line is noise.

There's a ritual that's become almost universal among developers adopting AI coding agents. You set up a new repo, run `/init`, watch the agent scan your codebase, and get back a shiny AGENTS.md. It describes your directory structure, your tech stack, your testing conventions. You skim it, it looks reasonable, you commit it. You feel like you've done the responsible thing. Your agent is configured.

Two papers published in early 2026 suggest you might have just made your agent slower, more expensive, and no more accurate. And the implications go further than whether to include a codebase overview.

Beyond what to put in it, there's a structural problem worth naming early: a single AGENTS.md at the root of your repo isn't sufficient for any codebase of real complexity. What you actually need is a hierarchy of AGENTS.md files - placed at the relevant directory or module level - automatically maintained so that each agent gets context scoped precisely to the code it's working in, rather than a monolithic file that conflates concerns across the entire project.

What the research actually says

Lulla et al. (ICSE JAWs 2026) ran a clean paired experiment: 124 real GitHub pull requests, executed with and without an AGENTS.md file, everything else held constant. Same task, same repo snapshot, same agent. They found that presence of an AGENTS.md file reduced median wall-clock runtime by 28.64% and output token consumption by 16.58%. Statistically significant. The agent got cheaper and faster.

That sounds like a clear win. Hold that thought.

A separate study out of ETH Zurich tested four agents across SWE-bench and a custom benchmark of repos that already had developer-authored context files. Their finding cuts the other way: LLM-generated context files reduced task success by 2-3% while increasing cost by over 20%. Developer-written files improved success by about 4% - but also increased cost by up to 19%.

So which is it? Does AGENTS.md help or hurt?

Both, depending on what you put in it. And that's actually the more interesting result.

The ETH Zurich paper discovered something that reframes the whole debate: when they stripped all documentation from the repos - READMEs, docs folders, markdown files - and *then* tested with LLM-generated context files, those files suddenly improved performance by 2.7%. The auto-generated content isn't useless. It's redundant. The agent could find all of it anyway by reading the repo. Hand it the same information twice and you've just added noise.

The Lulla paper, by contrast, used human-authored AGENTS.md files from repos where developers had actually been maintaining them. Real project-specific knowledge. Non-obvious tooling requirements. Actual gotchas. That's the context that saves the agent time because it doesn't have to infer what it can't discover.

The question isn't whether to have an AGENTS.md. The question is what earns a line in it.

The pink elephant problem

There's a subtler cost to AGENTS.md files that doesn't show up in efficiency metrics: the anchoring effect.

If your AGENTS.md mentions that you use tRPC on the backend - even just as a passing note in an architecture overview - the model now has tRPC in context for every single prompt. If tRPC is only used in a handful of legacy endpoints and everything new runs on something else, you've just biased your agent toward the wrong pattern for the current codebase. You said it, it's there, and LLMs don't distinguish between "this is what we used to do" and "this is what you should do."

This is the same reason prompt engineering wisdom has always said to be careful about what you put in your system prompt. Everything you add competes with the actual task. Research on LLM context more broadly shows a consistent pattern - more context often degrades performance, not just through cost but through diluted attention. Liu et al.'s "Lost in the Middle" result (2024) showed LLMs struggle with information placed in the middle of long contexts. Levy et al. showed that longer context degrades task performance even when the content is perfectly relevant. Every line in AGENTS.md is a line competing with the thing you actually asked the agent to do.

The case against /init specifically

Here's the core problem with auto-generated context files. What does `/init` produce? Codebase overviews. Directory structure. Tech stack description. Module explanations. The ETH Zurich paper found that 100% of Sonnet 4.5's auto-generated context files contained codebase overviews. 99% of GPT-5.2's did the same.

These are precisely the things an agent can discover on its own by listing directories and reading your existing READMEs - which it does anyway, regardless of whether AGENTS.md exists. So you've added a file that the agent reads, then goes and confirms by reading your actual code, and now has to reconcile two sources of truth. More reasoning tokens. More steps. Same outcome, except slower and more expensive.

What actually earns a line

The ETH Zurich paper showed something concrete about what works. When a developer-written context file mentioned `uv`, agents used it 1.6 times per task on average. When it wasn't mentioned, agents used it fewer than 0.01 times. The same pattern held for other repo-specific tools: 2.5 uses per task when mentioned, versus fewer than 0.05 when not.

`uv` versus `pip` is a perfect example of what belongs in AGENTS.md. It's:

- Not discoverable from the codebase without inference
- Operationally significant (it changes the commands the agent runs)
- Impossible for the agent to guess correctly by convention

Compare that to "this project uses a monorepo structure with packages in /packages" - something the agent finds in the first directory listing it runs. One of these earns a line. One of these is noise.

The practical filter is simple: can the agent discover this on its own by reading your code? If yes, delete it. Every line should represent information that isn't already in the repo.

That means your AGENTS.md should look more like:

- Use `uv` for package management
- Always run tests with `–no-cache` or you'll get false positives from the fixture setup
- The auth module uses a custom middleware pattern; do not refactor to standard Express middleware
- The `legacy/` directory is deprecated but imported by three production modules - don't delete anything in it

And almost nothing else.

The static file problem

Even if you write a tight, non-redundant AGENTS.md, it has a structural weakness: it's static, but tasks are dynamic.

Your file says "always run the test suite before committing." The agent is doing a

documentation change. It faithfully runs the full test suite. Tokens burned, minutes wasted, on an instruction that made sense for code changes but not for this one.

This isn't a hypothetical failure mode. It's baked into the architecture. A flat instruction set can't condition on what kind of task is being run. An agent working on a CSS refactor doesn't need your database migration warnings. One implementing a security fix probably should skip the performance optimization hints. A single monolithic file gets loaded the same way every time.

The ACE framework (Agentic Context Engineering, ICLR 2026) tackles this directly by treating context as an evolving playbook that adapts through a generator/reflector/curator pipeline, rather than a static file. On agent benchmarks, it outperformed static approaches by 12.3%. The architecture matters.

The better architecture people aren't building yet

Several people in the AGENTS.md discourse have converged on roughly the same idea independently: the right structure isn't a monolithic file, it's a routing layer with focused context loaded on demand.

The structure would look something like:

Layer 1: Protocol file (what AGENTS.md should actually be) Not a codebase overview. Not a style guide. A routing document. Available personas and when to invoke them. Available skills and what task classes they cover. Available MCP connections and what they're for. The minimum essential repo facts that the agent genuinely cannot discover - and nothing else.

Layer 2: Focused persona/skill files Each loaded selectively based on task type. A UX-focused agent working on component architecture loads different context than a backend agent debugging a data pipeline. Neither loads the other's context. The total context for any given task stays bounded.

Layer 3: A maintenance subagent Whose only job is keeping the protocol file accurate as the codebase evolves. Because here's the thing nobody talks about: documentation rots. The ETH Zurich study used freshly-generated context files and

still found degraded performance. Real-world AGENTS.md files that haven't been touched in six months - describing a dependency you've since replaced, a directory structure that's changed - are going to be worse. Much worse.

None of the major coding agents expose the lifecycle hooks to make this architecture easy to build. You can approximate it with sub-agents and scoped context, but there's no clean solution yet. That's a tooling gap waiting to be filled.

The automated optimization angle

The most provocative finding in this space comes from Arize AI's prompt learning work. Instead of manually writing CLAUDE.md instructions, they used an automated optimization loop - run the agent on training tasks, evaluate output, generate LLM feedback on why solutions failed, use meta-prompting to refine the instructions. Repeat.

Results: +5.19% accuracy improvement on a cross-repo test split. +10.87% on an in-repo split (training on past Django issues, testing on future ones).

What the optimizer discovered is the uncomfortable implication: what helps a human understand a codebase and what helps an LLM navigate it are often different things. An instruction that seems obviously useful to you - "this service uses the repository pattern" - might be noise to the model. And something the model actually needs - some particular import path disambiguation, some non-obvious file naming convention - might never occur to you to write down because you've internalized it.

The manual approach to AGENTS.md assumes that you know what the agent needs. The empirical evidence suggests you probably don't. The optimizer figures out the delta between what you think matters and what actually moves the needle.

This doesn't mean you should give up on writing context files - the automated optimization research is still early, and 5% is meaningful but not transformative. It means you should hold your intuitions about what to include loosely, and probably test them.

The right mindset for AGENTS.md

Think of AGENTS.md as a living document of friction you haven't fixed yet.

Every line you add is a signal about something in your codebase that's confusing enough to trip an AI agent - which means it's probably confusing enough to trip a new human contributor too. The right response to that signal isn't to grow the context file. It's to fix the actual problem.

Agent keeps putting new utilities in the wrong directory? Maybe the directory structure is confusing and should be reorganized. Agent keeps reaching for a deprecated dependency? Maybe the import structure makes the wrong one too easy to grab. Agent forgetting to run type checks? Maybe the build pipeline should catch that automatically instead of relying on a prose instruction.

The AGENTS.md becomes a diagnostic tool rather than a permanent configuration. You add a line, you investigate why the agent keeps making this mistake, you fix the underlying thing, and then you can probably delete the line.

One technique worth trying: start your AGENTS.md nearly empty, and add a single instruction: "If you encounter something surprising or confusing in this project, flag it as a comment." Most of the agent's proposed additions aren't things you want to keep permanently - they're indicators of where the codebase is unclear. Fix those. Keep the file minimal.

What the research doesn't resolve

Both papers have real limitations worth naming.

The Lulla paper doesn't measure correctness at all - only efficiency. You can't conclude from it that AGENTS.md improves output quality. The authors are clear about this: they performed a sanity check to confirm agents produced non-trivial output, but a full correctness evaluation is future work. Faster and cheaper is meaningful, but not if the code is wrong.

The ETH Zurich paper used fresh context files, which is ideal for experimental control

but arguably tests a scenario most developers don't actually live in. The real-world case is context files that have been in the repo for months and are partially out of date. That should perform worse than what the paper measured.

Neither paper tests the hierarchical, dynamically-loaded approach that multiple practitioners argue is the correct architecture. The "flat monolithic context file" is the only thing being evaluated. Whether a properly layered system with selective context loading performs differently is still an open empirical question.

And there's the model trajectory problem: every few months, developers report needing less in their context files because the underlying models have gotten better at codebase navigation. Instructions that were essential six months ago become redundant as models improve. The AGENTS.md you write today might be pure overhead by next quarter.

Practical takeaways

Stop running `/init`. The auto-generated output is redundant with your existing documentation and adds overhead without benefit, unless your repo has genuinely zero documentation - in which case it's marginally better than nothing.

Before adding any line to AGENTS.md, ask: can the agent find this by reading the code? If yes, don't write it.

When an agent struggles with something repeatedly, treat it as a codebase problem before treating it as a context problem. Restructure the code. Add a linter rule. Improve the test coverage. Reach for AGENTS.md only after you've exhausted those options.

If you're running agents at scale in CI/CD or automated review pipelines, the 15-20% cost overhead from context files compounds across thousands of runs. Calculate it before assuming the tradeoff is worth it.

Consider building a maintenance agent whose job is keeping the context file accurate rather than letting it rot.

And hold your intuitions about what the agent needs loosely. What seems obviously

useful to you might be noise to the model. The empirical evidence suggests the gap between what we think agents need and what actually helps them is substantial - and probably not in the direction we'd expect.

The instinct to onboard your coding agent like a new hire - give it the office tour, explain the org chart, walk it through the architecture - comes from a reasonable place. But coding agents aren't new hires. They can grep the entire codebase before you finish typing your prompt. What they need isn't a map. They need to know where the landmines are.

And maybe, increasingly, they don't even need that.

If you are interested in video, Theo's take on this topic is excellent too.

Watch on X

Addy Osmani is a Software Engineer at Google working on Google Cloud and Gemini.

Tweet Bluesky Mastodon Threads LinkedIn Share

Want more? Subscribe to my free newsletter:

example@gmail.com Subscribe